Contents

CHAPTER 1

Introduction to Polynomials

Watch Khan Academy's **Polynomials intro** video at https://youtu.be/Vm7H0VTlIco

A *monomial* is the product of a number and a variable raised to a non-negative (but possibly zero) integer power. Here are some examples of monomials:

$3x^2$ πx^2 $7x$ $-\frac{2}{3}x^{12}$

$-2x^{15}$ $(3.33)x^{100}$ 3 0

The exponent is called the *degree* of the monomial. For example, $3x^{17}$ has degree 17, $-7x$ has degree 1, and 3.2 has degree 0 (because you can think of it as $(3.2)x^0$).. The exponent cannot be x (or any non-constant variable), as that becomes an exponential function rather than polynomial.[1]

The number in the product is called the *coefficient*. Example: $3x^{17}$ has a coefficient of 3, $-2x$ has a coefficient of -2, and $(3.4)x^{1000}$ has a coefficient of 3.4.

A *polynomial* is the sum of one or more monomials. Here are some polynomials:

$4x^2 + 9x + 3.9$ $\pi x^2 + \pi x + \pi$ $7x + 2$

$-2x^{10} + (3.4)x - 45x^{900} - 1$ 3.3 $3x^{20}$

We say that each monomial is a *term* of the polynomial.

$x^{-5} + 12$ is *not* a polynomial because the first term has a negative exponent.

$x^2 - 32x^{\frac{1}{2}} + x$ is *not* a polynomial because the second term has a non-integer exponent.

$\frac{x+2}{x^2+x+5}$ is *not* a polynomial because it is not just a sum of monomials.

[1] A quadratic is a polynomial with a maximum degree of 2

Exercise 1 Identifying Polynomials

Circle only the polynomials.

Working Space

$-2x^3+\frac{1}{2}x+3.9(4.5)x^2+\pi x$ $\qquad 7$

$x^{\frac{2}{3}}$ $\qquad 3x^{20}+2x^{19}-5x^{18}$

$2x^{-10}+4x-1$

Answer on Page 57

We typically write a polynomial starting at the term with the highest degree and proceed in decreasing order to the term with the lowest degree:

$$2x^9-3x^7+\frac{3}{4}x^3+x^2+\pi x-9.3$$

This is known as *the standard form*. The first term of the standard form is called *the leading term*, and we often call the coefficient of the leading term *the leading coefficient*. We sometimes speak of the degree of the polynomial, which is just the degree of the leading term.

Exercise 2 Standard of a Polynomial

Write $21x^2-x^3+\pi-1000x$ in standard form. What is the degree of this polynomial? What is its leading coefficient?

Working Space

Answer on Page 57

Exercise 3 Evaluate a Polynomial

Let $y = x^3 - 3x^2 + 10x - 12$. What is y when x is 4?

Working Space

Answer on Page 57

We would be remiss in our duties if we didn't mention one more thing about polynomials: Mathematicians have defined a polynomial to be a sum of a *finite* number of monomials.

It is certainly possible to have a sum of an infinite number of monomials like this:

$$1 + \frac{1}{2}x + \frac{1}{4}x^2 + \frac{1}{8}x^3 + \frac{1}{16}x^4 + \dots$$

This is an example of an *infinite series*, which we don't consider polynomials. Infinite series are interesting and useful, but we will not discuss them in detail until later in the course.

Chapter 2

Python Lists

Watch CS Dojo's **Introduction to Lists in Python** video at https://www.youtube.com/watch?v=tw7ror9x32s

To review, Python list is an indexed collection. The indices start at zero. You can create a list using square brackets.

You are now going to write a program that makes an array of strings. Type this code into a file called `faves.py`:

```
favorites = ["Raindrops", "Whiskers", "Kettles", "Mittens"]
favorites.append("Packages")
print("Here are all my favorites:", favorites)
print("My most favorite thing is", favorites[0])
print("My second most favorite is", favorites[1])
number_of_faves = len(favorites)
print("Number of things I like:", number_of_faves)

for i in range(number_of_faves):
    print(i, ": I like", favorites[i])
```

Run it:

```
$ python3 faves.py
Here are all my favorites: ['Raindrops', 'Whiskers', 'Kettles', 'Mittens',
↪  'Packages']
My most favorite thing is Raindrops
My second most favorite is Whiskers
Number of things I like: 5
0 : I like Raindrops
1 : I like Whiskers
2 : I like Kettles
3 : I like Mittens
4 : I like Packages
```

After you have run the code, study it until the output makes sense.

Exercise 4 Assign into list

Before you list the items, replace "Mittens" with "Gloves".

Working Space

Answer on Page 57

2.1 Evaluating Polynomials in Python

First, before you go any further, you need to know that raising a number to a power is done with ** in Python. For example, to get 5^2, you would write `5**2`.

Back to polynomials: if you had a polynomial like $2x^3 - 9x + 12$, you could write it like this: $12x^0 + (-9)x^1 + 0x^2 + 2x^3$. We could use this representation to keep a polynomial in a Python list. We would simply store all the coefficients in order:

```
1 pn1 = [12,-9,0,2]
```

In the list, the index of each coefficient would correspond to the degree of that monomial. For example, in the list, 2 is at index 3, so that entry represents $2x^3$.

In the last chapter, you evaluated the polynomial $x^3 - 3x^2 + 10x - 12$ at $x = 4$. Now you will write code that does that evaluation. Create a file called `polynomials.py` and type in the following:

```
1  def evaluate_polynomial(pn, x):
2      sum = 0.0
3      for degree in range(len(pn)):
4          coefficient = pn[degree]
5          term_value = coefficient * x ** degree
6          sum = sum + term_value
7      return sum
8
9  pn1 = [-12.0, 10.0, -3.0, 1.0]
10 y = evaluate_polynomial(pn1, 4.0)
11 print("Polynomial 1: When x is 4.0, y is", y)
```

Run it. It should evaluate to 44.0.

2.2 Walking the list backwards

Now you are going to make a function that makes a pretty string to represent your polynomial. Here is how it will be used:

```
def polynomial_to_string(pn):
    ...Your Code Here...

pn_test = [-12.0, 10.0, 0.0, 1.0]
print(polynomial_to_string(pn1))
```

This would output:

```
1.0x**3 + 10.0x + -12.0
```

This is not as simple as you might hope. In particular:

- You should skip the terms with a coefficient of zero
- The term of degree 1 has an x, but no exponent
- The term of degree 0 has neither an x nor an exponent
- Standard form demands that you list the terms in the reverse order from that of your coefficients list. You will need to walk the list from last to first.

Add this function to your `polynomials.py` file after your `evaluate_polynomial` function:

```
def polynomial_to_string(pn):

    # Make a list of the monomial strings
    monomial_strings = []

    # Start at the term with the largest degree
    degree = len(pn) - 1

    # Go through the list backwards stop after constant term
    while degree >= 0:
        coefficient = pn[degree]

        # Skip any term with a zero coefficient
        if coefficient != 0.0:

            # Describe the monomial
            if degree == 0:
                monomial_string = "{}".format(coefficient)
            elif degree == 1:
```

```
                monomial_string = "{}x".format(coefficient)
            else:
                monomial_string = "{}x^{}".format(coefficient, degree)

            # Add it to the list
            monomial_strings.append(monomial_string)

        # Move to the previous term
        degree = degree - 1

    # Deal with the zero polynomial
    if len(monomial_strings) == 0:
        monomial_strings.append("0.0")

    # Make a string that joins the terms with a plus sign
    return " + ".join(monomial_strings)
```

Note that in a list n items, the indices go from 0 to $n-1$. When we are walking the list backwards, we start at `len(pn) - 1` and stop at zero.

Look over the code and google the functions you aren't familiar with. For example, if you want to know about the (join) function, google for "python join".

Now, change your code to use the new function:

```
pn1 = [-12.0, 10.0, -3.0, 1.0]
y = evaluate_polynomial(pn1, 4.0)
print("y =", polynomial\_to\_string(pn1))
print("    When x is 4.0, y is", y)
```

Run the program. Does the function work?

Exercise 5 Evaluate Polynomials

Using the function that you just wrote, add a few lines of code to `polynomials.py` to evaluate the following polynomials:

- Find $4x^4 - 7x^3 - 2x^2 + 5x + 2.5$ at $x = 8.5$. It should be 16481.875
- Find $5x^5 - 9$ at $x = 2.0$. It should be 151.0

Working Space

Answer on Page 57

2.3 Plot the polynomial

We can evaluate a polynomial at many points and plot them on a graph. You are going to write the code to do this. Create a new file called `plot_polynomial.py`. Copy your `evaluate_polynomial` function into the new file.

Add a line at the beginning of the program that imports the plotting library matplotlib:

```
1 import matplotlib.pyplot as plt
```

After the `evaluate_polynomial` function:

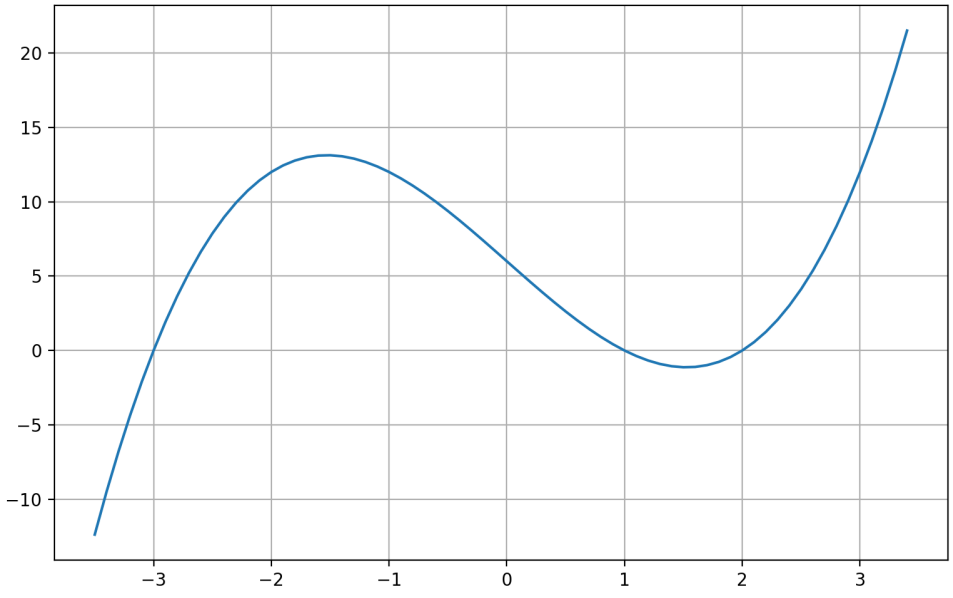
- Create a list with polynomial coefficients.
- Create two empty arrays, one for x values and one for y values.
- Fill the x array with values from -3.5 to 3.5. Evaluate the polynomial at each of these points; put those values in the y array.
- Plot them

Like this:

```
1 # x**3 - 7x + 6
2 pn = [6.0, -7.0, 0.0, 1.0]
3
```

```
# These lists will hold our x and y values
x_list = []
y_list = []

# Start at x=-3.5
current_x =-3.5

# End at x=3.5
while current_x <= 3.5:
    current_y = evaluate_polynomial(pn, current_x)

    # Add x and y to respective lists
    x_list.append(current_x)
    y_list.append(current_y)

    # Move x forward
    current_x += 0.1

# Plot the curve
plt.plot(x_list, y_list)
plt.grid(True)
plt.show()
```

You should get a beautiful plot like this:



If you received an error that the matplotlib was not found, use pip to install it:

```
1 $ pip3 install matplotlib
```

Exercise 6 Observations

Where does your polynomial cross the y-axis? Looking at the polynomial $x^3 - 7x+6$, could you have guessed that value?

Working Space

Where does your polynomial cross the x-axis? The places where a polynomial crosses the x-axis are called *its roots*. Later in the course, you will learn techniques for finding the roots of a polynomial.

Answer on Page 58

Chapter 3

Adding and Subtracting Polynomials

Watch Khan Academy's **Adding polynomials** video at https://youtu.be/ahdKdxsTj8E

When adding two monomials of the same degree, you sum their coefficients:

$$7x^3 + 4x^3 = 11x^3$$

Using this idea, when adding two polynomials, you convert them into one long polynomial, then simplify by combining terms with the same degree. For example:

$$\begin{aligned}(10x^3 - 2x + 13) + (-5x^2 + 7x - 12) \\ &= 10x^3 + (-2)x + 13 + (-5)x^2 + 7x + (-12) \\ &= 10x^3 + (-5)x^2 + (-2+7)x + (13-12) \\ &= 10x^3 - 5x^2 + 5x + 1\end{aligned}$$

Exercise 7 Adding Polynomials Practice

Working Space

Add the following polynomials:

1. $2x^3 - 5x^2 + 3x - 9$ and $x^3 - 2x^2 - 2x - 9$

2. $3x^5 - 5x^3 + 3x^2 - x - 3$ and $2x^4 - 2x^3 - 2x^2 + x - 9$

Answer on Page 58

Notice that in the second question, the degree 1 term disappears completely: $(-x) + x = 0$

One more tricky thing can happen — sometimes the coefficients don't add nicely. For example:

$$\pi x^2 - 3x^2 = (\pi - 3)x^2$$

That is as far as you can simplify it.

3.1 Subtraction

Now, watch Khan Academy's **Subtracting polynomials** at https://youtu.be/5ZdxnFspyP8.

When subtracting one polynomial from the other, it is a lot like adding two polynomials. The difference: when making the two polynomials into one long polynomial, we multiply each monomial that is being subtracted by -1. For example:

$$\begin{aligned}(2x^2 - 3x + 9) - (5x^2 - 7x + 4) &= 2x^2 + (-3)x + 9 + (-5)x^2 + 7x + (-4) \\ &= (2-5)x^2 + (-3+7)x + (9-4) \\ &= -3x^2 + 4x + 5\end{aligned}$$

Exercise 8 Subtracting Polynomials Practice

Working Space

Subtract the following polynomials:

1. $(2x^3 - 5x^2 + 3x - 9) - (x^3 - 2x^2 - 2x - 9)$

2. $(3x^5 - 5x^3 + 3x^2 - x - 3) - (2x^4 - 2x^3 - 2x^2 + x - 9)$

Answer on Page 58

3.2 Adding Polynomials in Python

As a reminder, in our Python code, we are representing a polynomial with a list of coefficients. The first coefficient is the constant term. The last coefficient is the leading coefficient. So, we can imagine $-5x^3 + 3x^2 - 4x + 9$ and $2x^3 + 4x^2 - 9$ would look like this: *FIXME: Diagram here*

To add the two polynomials then, we sum the coefficients for each degree. *FIXME: Diagram here*

Create a file called `add_polynomials.py`, and type in the following:

```
1  def add_polynomials(a, b):
2      degree_of_result = len(a)
3      result = []
4      for i in range(degree_of_result):
5          coefficient_a = a[i]
6          coefficient_b = b[i]
7          result.append(coefficient_a + coefficient_b)
8      return result
9
10 polynomial1 = [9.0, -4.0, 3.0, -5.0]
```

```
polynomial2 = [-9.0, 0.0, 4.0, 2.0]
polynomial3 = add_polynomials(polynomial1, polynomial2)

print('Sum =', polynomial3)
```

Run the program.

Unfortunately, this code only works if the polynomials are the same length. For example, try making `polynomial1` have a larger degree than `polynomial2`:

```
# x**4 - 5x**3 + 3x**2 - 4x + 9
polynomial1 = [9.0, -4.0, 3.0, -5.0, 1.0]

# 2x**3 + 4x**2 - 9
polynomial2 = [-9.0, 0.0, 4.0, 2.0]
polynomial3 = add_polynomials(polynomial1, polynomial2)
print('Sum =', polynomial3)
```

See the problem?

Exercise 9 Dealing with polynomials of different degrees

Working Space

Can you fix the function `add_polynomials` to handle polynomials of different degrees?

Here is a hint: In Python, there is a `max` function that returns the largest of the numbers it is passed.

```
biggest = max(5,7)
```

Here `biggest` would be set to 7.

Here is another hint: If you have an array `mylist`, `i`, a non-negative integer, is only a legit index if `i < len(mylist)`.

Answer on Page 58

3.3 Scalar multiplication of polynomials

If you multiply a polynomial with a number, the distributive property applies:

$$(3.1)(2x^2 + 3x + 1) = (6.2)x^2 + (9.3)x + 3.1$$

(When we are talking about things that are more complicated than a number, we use the word *scalar* to mean "Just a number". This is the product of a scalar and a polynomial.)

In `add_polynomials.py`, add a function to that multiplies a scalar and a polynomial:

```
def scalar_polynomial_multiply(s, pn):
    result = []
    for coefficient in pn:
        result.append(s * coefficient)
    return result
```

Somewhere near the end of the program, test this function:

```
polynomial4 = scalar_polynomial_multiply(5.0, polynomial1)
print('Scalar product =', polynomial_to_string(polynomial4))
```

Exercise 10 Subtract polynomials in Python

Now, implement a function that does subtraction using `scalar_polynomial_multiply` and `add_polynomials`.

Working Space

It should look like this:

```
def subtract_polynomial(a, b):
    ...Your code here...

polynomial5 = [9.0, -4.0, 3.0, -5.0]
polynomial6 = [-9.0, 0.0, 4.0, 2.0, 1.0]
polynomial7 = subtract_polynomial(polynomial5, polynomial6)
print('Difference =', polynomial_to_string(polynomial7))
```

Answer on Page 59

CHAPTER 4

Multiplying Polynomials

Watch Khan Academy's **Multiplying monomials** at https://www.youtube.com/watch?v=7EplN4Ch98Q.

To review, when you multiply two monomials, you take the product of their coefficients and the sum of their degrees:

$$(2x^6)(5x^3) = (2)(5)(x^6)(x^3) = 10x^9$$

If you have a product of more than two monomials, multiply *all* the coefficients and sum *all* the exponents:

$$(3x^2)(2x^3)(4x) = (3)(2)(4)(x^2)(x^3)(x^1) = 24x^6$$

Exercise 11 Multiplying monomials

Working Space

Multiply these monomials:

1. $(3x^2)(5x^3)$

2. $(2x)(4x^9)$

3. $(-5.5x^2)(2x^3)$

4. $(\pi)(-2x^5)$

5. $(2x)(3x^2)(5x^7)$

Answer on Page 59

4.1 Multiplying a monomial and a polynomial

Watch Khan Academy's **Multiplying monomials by polynomials** at https://youtu.be/pD2-H15ucNE. When multiplying a monomial and a polynomial, you use the distributive property. After that, it is just multiplying several pairs of monomials:

$$(3x^2)(4x^3 - 2x^2 + 3x - 7)$$
$$= (3x^2)(4x^3) + (3x^2)(-2x^2) + (3x^2)(3x) + (3x^2)(-7)$$
$$= 12x^5 - 6x^4 + 9x^3 - 21x^2$$

Exercise 12 Multiplying a monomial and a polynomial

Working Space

Multiply these monomials:

1. $(3x^2)(5x^3 - 2x + 3)$

2. $(2x)(4x^9 - 1)$

3. $(-5.5x^2)(2x^3 + 4x^2 + 6)$

4. $(\pi)(-2x^5 + 3x^4 + x)$

5. $(2x)(3x^2)(5x^7 + 2x)$

Answer on Page 59

4.2 Multiplying polynomials

Watch Khan Academy's **Multiplying binomials by polynomials** video at https://youtu.be/D6mivA_8L8U

When you are multiplying two polynomials, you will use the distributive property several times to make it one long polynomial. You will then combine the terms with the same degree. For example,

$$\begin{aligned}(2x^2-3)(5x^2+2x-7)&\\ &= (2x^2)(5x^2+2x-7)+(-3)(5x^2+2x-7)\\ &= (2x^2)(5x^2)+(2x^2)(2x)+(2x^2)(-7)+(-3)(5x^2)+(-3)(2x)+(-3)(-7)\\ &= 10^4+4x^3+-14x^2+-15x^2+-6x+21 = 10^4+4x^3+-29x^2+-6x+21\end{aligned}$$

One common form that you will see is multiplying two binomials together:

$$(2x+7)(5x+3) = (2x)(5x+3)+(7)(5x+3) = (2x)(5x)+(7)(5x)+(2x)(3)+(7)(3)$$

Notice the product has become the sum of four parts: the firsts, the inners, the outers, and the lasts. People sometimes use the mnemonic FOIL to remember this pattern, but there is a general rule that works for all product of polynomials, not just binomials: Every term in the first will be multiplied by every term in the second, and then just add them together.

So, for example, if you have a polynomial s with three terms and you multiply it by a polynomial t with five terms, you will get a sum of 15 terms — each term is a product of two monomials, one from s and one from t. (Of course, several of those terms might have the same degree, so they will be combined together when you simplify. Thus you typically end up with a polynomial with less than 15 terms.)

Using this rule, here is how you would multiply $2x^2-3x+1$ and $5x^2+2x-7$:

$$\begin{aligned}(2x^2-3x+1)(5x^2+2x-7) &= \begin{array}{rcrcrc}(2x^2)(5x^2) & + & (2x^2)(2x) & + & (2x^2)(-7) & +\\ (-3x)(5x^2) & + & (-3x)(2x) & + & (-3x)(-7) & +\\ (1)(5x^2) & + & (1)(2x) & + & (1)(-7) & \end{array}\\ &= 10x^4+4x^3+(-14)x^2+(-15)x^3+(-6)x^2+21x+5x^2+2x+(-7)\\ &= 10x^4+(4-15)x^3+(-14-6+5)x^2+(21+2)x+(-7)\\ &= 10x^4-11x^3-15x^2+23x-7\end{aligned}$$

Note that the product (before combining terms with the same degree) has $3\times 3 = 9$ terms — every possible combination of a term from the first polynomial and a term from the second polynomial.

One common source of error is losing track of the negative signs. You will need to be really careful. We have found that it helps to use + between all terms, and use negative coefficients to express subtraction. For example, if the problem says $4x^2-5x-3$, you should work with that as $4x^2+(-5)x+(-3)$.

Exercise 13 Multiplying polynomials

Multiply the following pairs of polynomials:

1. $2x + 1$ and $3x - 2$

2. $-3x^2 + 5$ and $4x - 2$

3. $-2x - 1$ and $-3x - \pi$

4. $-2x^5 + 5x$ and $3x^5 + 2x$

Working Space

Answer on Page 59

Exercise 14 Observations

Let's say you have two polynomials, p_1 and p_2. p_1 has degree 23. p_2 has degree 12. What is the degree of their product?

Working Space

Answer on Page 60

Chapter 5

Multiplying Polynomials in Python

At this point, you have created a nice toolbox of functions for dealing with lists of coefficients as polynomials. Create a file called `poly.py` and copy the following functions into it:

- `evaluate_polynomial`
- `polynomial_to_string`
- `add_polynomials`
- `scalar_polynomial_multiply`
- `subtract_polynomial`

Now, create another file in the same directory called `test.py`. Type this into that file:

```
import poly

polynomial_a = [9.0, -4.0, 3.0, -5.0]
print('Polynomial A =', poly.polynomial_to_string(polynomial_a))

polynomial_b = [-9.0, 0.0, 4.0, 2.0, 1.0]
print('Polynomial B =', poly.polynomial_to_string(polynomial_b))

# Evaluation
value_of_b = poly.evaluate_polynomial(polynomial_b, 3)
print('Polynomial B at 3 =', value_of_b)

# Adding
a_plus_b = poly.add_polynomials(polynomial_a, polynomial_b)
print('A + B =', poly.polynomial_to_string(a_plus_b))

# Scalar multiplication
b_scalar = poly.scalar_polynomial_multiply(-3.2, polynomial_b)
print('-3.2 * Polynomial B =', poly.polynomial_to_string(b_scalar))

# Subtraction
a_minus_b = poly.subtract_polynomial(polynomial_a, polynomial_b)
print('A - B =', poly.polynomial_to_string(a_minus_b))
```

When you run it, you should get the following:

```
Polynomial A = -5.0x^3 + 3.0x^2 + -4.0x + 9.0
Polynomial B = 1.0x^4 + 2.0x^3 + 4.0x^2 + -9.0
Polynomial B at 3 = 162.0
A + B = 1.0x^4 + -3.0x^3 + 7.0x^2 + -4.0x
-3.2 * Polynomial B = -3.2x^4 + -6.4x^3 + -12.8x^2 + 28.8
A - B = -1.0x^4 + -7.0x^3 + -1.0x^2 + -4.0x + 18.0
```

You are now ready to implement the multiplication of polynomials. The function will look like this:

```
def multiply_polynomials(a, b):
  ...Your code here...
```

It will return a list of coefficients.

In an exercise in the last chapter, you were asked " Let's say I have two polynomials, p_1 and p_2. p_1 has degree 23. p_2 has degree 12. What is the degree of their product?" The answer was $23 + 12 = 35$.

In our implementation, a polynomial of degree 23 is held in a list of length 24.

In Python, we will be trying to multiply a polynomial `a` and a polynomial `b` represented as lists. What is the degree of that product?

```
result_degree = (len(a) - 1) + (len(b) - 1)
```

Now, we need to create an array of zeros that is one longer than that. Here is a cute Python trick: If you have a list, you can replicate it using the * operator.

```
a = [5,7]
b = a * 4
print(b)
# [5, 7, 5, 7, 5, 7, 5, 7]
```

Here iss how you will get a list of zeros:

```
result = [0.0] * (result_degree + 1)
```

We will step through `a`, getting the index and value of each entry. You can do this in one

line using `enumerate`:

```
for a_degree, a_coefficient in enumerate(a):
```

For each of those, we will step through the entire b polynomial. As you multiply together each term, you will add it to the appropriate coefficient of the result.

Here is the whole function:

```
def multiply_polynomials(a, b): # What is the degree of the resulting
polynomial?  result_degree = (len(a) - 1) + (len(b) - 1)

    # Make a list of zeros to hold the coefficients result = [0.0] *
    (result_degree + 1)

    # Iterate over the indices and values of a for a_degree,
    a_coefficient in enumerate(a):

        # Iterate over the indices and values of b for b_degree,
        b_coefficient in enumerate(b):

            # Calculate the resulting monomial coefficient =
            a_coefficient * b_coefficient degree = a_degree + b_degree

            # Add it to the right bucket
            result[degree] = result[degree] + coefficient

    return result
```

Take a long look at that function. When you understand it, type it into `poly.py`.

In `test.py`, try out the new function:

```
# Multiplication
a_times_b = poly.multiply_polynomials(polynomial_a, polynomial_b)
print('A x B =', poly.polynomial_to_string(a_times_b))
```

This is an example of a *nested loop*. The outer loop steps through the polynomial a. For each step it takes, the inner loop steps through the entire polynomial b.

5.1 Something surprising about lists

You can imagine that you might want to create two very similar polynomials. Let's say polynomial c is $x^2 + 2x + 1$ and polynomial d is $x^2 - 2x + 1$. You might think you are very

clever to just alter that degree 1 coefficient like this:

```
c = [1.0, 2.0, 1.0]
d = c
d[1] = -2.0
```

If you printed out c, you would get $[1.0, -2.0, 1.0]$. Why? You assigned two variables (c and d) to the *the same list*. So, when you use one reference (d) to change the list, you see the change if you look at the list from either reference. *FIXME: Diagram of two references to the same list here.*

To create two separate lists, you would need to explicitly make a copy:

```
c = [1.0, 2.0, 1.0]
d = c.copy()
d[1] = -2.0
```

Chapter 6

Differentiating Polynomials

If you had a function that gave you the height of an object, it would be handy to be able to figure out a function that gave you the velocity at which it was rising or falling. The process of converting the position function into a velocity function is known as *differentiation* or *finding the derivative*. There are a bunch of rules for finding a derivative, but differentiating polynomials only requires three:

- The derivative of a sum is equal to the sum of the derivatives.
- The derivative of a constant is zero.
- The derivative of a nonconstant monomial at^b (a and b are constant numbers, t is time) is abt^{b-1}. This is referred to as the **power rule**.

Power Rule

For $f(x) = ax^n$, the derivative is $f'(x) = anx^{n-1}$.

Say, for example, that we tell you that the height (position) in meters of a quadcopter at second t is given by $2t^3 - 5t^2 + 9t + 200$. You could tell us that its vertical velocity is $6t^2 - 10t + 9$.

We indicate the derivative of a function with an apostrophe (read as "prime") between the name of the function and the variable. For example, the derivative of $h(t)$ is $h'(t)$ (which is read out loud as "h prime of t").

Exercise 15 Differentiation of polynomials

Differentiate the following polynomials.

Working Space

1. $f(t) = 2t^3 - 3t^2 - 4t$
2. $g(t) = 2t^{-3/4}$
3. $F(r) = \frac{5}{r^3}$
4. $H(u) = (3u - 1)(u + 2)$

Answer on Page 60

Notice that the degree of the derivative is one less than the degree of the original polynomial. (Unless, of course, the degree of the original is already zero.)

Now, if you know that a position is given by a polynomial, you can differentiate it to find the object's velocity at any time.

The same trick works for acceleration: Let's say you know a function that gives an object's velocity. To find its acceleration at any time, you take the derivative of the velocity function (the second derivative).

Exercise 16 Differentiation of polynomials in Python

Write a function that returns the derivative of a polynomial in `poly.py`. It should look like this:

```
def derivative_of_polynomial(pn):
  ...Your code here...
```

When you test it in `test.py`, it should look like this:

```
# 3x**3 + 2x + 5
p1 = [5.0, 2.0, 0.0, 3.0]
d1 = poly.derivative_of_polynomial(p1)
# d1 should be 9x**2 + 2
print("Derivative of", poly.polynomial_to_string(p1),"is",
    poly.polynomial_to_string(d1))

# Check constant polynomials
p2 = [-9.0]
d2 = poly.derivative_of_polynomial(p2)
# d2 should be 0.0
print("Derivative of", poly.polynomial_to_string(p2),"is",
    poly.polynomial_to_string(d2))
```

Working Space

Answer on Page 60

6.1 Second order and higher derivatives

As seen from the example, with height, velocity, and acceleration, you can take the derivative of a derivative, which is called the second derivative and is indicated with two marks, like so:

$$\frac{d}{dx}f'(x) = f''(x)$$

When you have the height function (or position function, in the case of horizontal motion) of an object, the first derivative describes the velocity of the object, and the second deriva-

tive describes the acceleration. Suppose the motion of a particle is given by $s(t) = t^3 - 5t$, where s is in meters and t is in seconds. What is the acceleration when the velocity is 0? First, we find the velocity function, $s'(t)$, and the acceleration function, $s''(t)$:

$$s'(t) = 3t^2 - 5$$

$$s''(t) = 6t$$

To find where the velocity is 0, set $s'(t) = 0$:

$$3t^2 - 5 = 0$$

$$3t^2 = 5$$

$$t^2 = \frac{5}{3}$$

$$t = \sqrt{\frac{5}{3}} \approx 1.29s$$

(We ignore the other solution, $t = -\sqrt{\frac{5}{3}}$ because it is usual for time to start at zero.)

Next, we use $t \approx 1.29s$ in the acceleration function, $s''(t)$:

$$s''(\sqrt{\frac{5}{3}}) = 6\sqrt{\frac{5}{3}} \approx 7.75\frac{m}{s^2}$$

For higher order derivatives, you just keep taking the derivative! So a third derivative is found by taking the derivative of the second derivative, and so on.

Exercise 17 Using Derivatives to Describe Motion

The position of a particle is described by the equation $s(t) = t^4 - 2t^3 + t^2 - t$, where s is in meters and t is in seconds.

(a) Find the velocity and acceleration as functions of t.

(b) Find the velocity after 1.5 s.

(c) Find the acceleration after 1.5 s.

(d) Is the object speeding up or slowing down at $t = 1.5$? How do you know?

Working Space

Answer on Page 61

Chapter 7

Python Classes

FIXME integrate this better with polynomials, but we need to cover the following

7.1 Object-Oriented Programming Introduction

Imagine you want to implement multiple dogs in python. It gets a bit complicated to do the following:

```
    dog1name = "Teddy"
    dog1age = 6
    dog1sound = "woof"

    dog2name = "Fluffers"
    dog2age = 2
    dog2sound = "bark"

    dog3name = "Bella"
    dog3age = 3
    dog3sound = "grr"

    print(f"{dog1name} is {dog1age} and says {dog1sound}!")
    print(f"{dog2name} is {dog2age} and says {dog2sound}!")
    print(f"{dog3name} is {dog3age} and says {dog3sound}!")


```

Instead, we can use classes, which are a way to create your own datatype. Classes can contain custom methods that either return, print, or calculate different values for you, and they can contain custom variables referred to as attributes.

```
class Dog:
    """A simple model of a dog."""
    # constructor - which creates a new model of a dog.
    def __init__(self, name: str, age: int, sound: str):
        self.name = name
        self.age = age
        self.sound = sound
    # method speak
    def speak(self) -> None:
```

```
        """Print a sentence describing the dog."""
        print(f"{self.name} is {self.age} and says {self.sound}!")

# create dog objects called "instances" with the given variables
dog1 = Dog("Teddy", 6, "woof")
dog2 = Dog("Fluffers", 2, "bark")
dog3 = Dog("Bella", 3, "grr")

# call the behavior on each object
dog1.speak()
dog2.speak()
dog3.speak()

```

Now let's talk about classes in the context of polynomials!

The built-in types, such as strings, have functions associated with them. So, for example, if you needed a string converted to uppercase, you would call its `upper()` function: -

```
my_string = "houston, we have a problem!"
louder_string = my_string.upper()
```

This would set `louder_string` to "HOUSTON, WE HAVE A PROBLEM!" When a function is associated with a datatype like this, it is called a *method*. A datatype with methods is known as a *class*. Creating a new version of a class in a variable is called an *instance*. For example, in the example, we would say "`my_string` is an instance of the class `str`. `str` has a method called `upper`"

The function `type` will tell you the type of any data:

```
print(type(my_string))
```

This will output

```
<class 'str'>
```

A class can also define operators. `+`, for example, is redefined by `str` to concatenate strings together:

```
long_string = "I saw " + "15 people"
```

7.2 Parent classes

Classes can have classes they inherit from (called "parent classes") or classes that inherit from them (called "child classes"). Parent classes (ie 'Animal') give a subclass (or child) different attributes or methods. Let's check out an example:

```
class Animal:
    """Generic animal base-class."""

    def __init__(self, name: str, age: int) -> None:
        self.name = name
        self.age = age

    def describe(self) -> None:
        """Print a basic description common to all animals."""
        print(f"{self.name} is {self.age} years old.")

class Dog(Animal):
    """Dog inherits name and age from Animal, adds its own sound."""

    def __init__(self, name: str, age: int, sound: str) -> None:
        super().__init__(name, age)      # initialize the Animal part
        self.sound = sound

    def speak(self) -> None:
        """Dog-specific implementation of speak()."""
        print(f"{self.name} is {self.age} and says {self.sound}!")


# demonstration
dogs = [
    Dog("Teddy", 6, "woof"),
    Dog("Fluffers", 2, "bark"),
    Dog("Bella", 3, "grr")
]

for dog in dogs:
    dog.describe()    # common behavior from Animal
    dog.speak()       # overridden behavior in Dog

```

Here, we have made a parent class for 'dog' called 'animal'.

7.3 Making a Polynomial class

You have created a bunch of useful python functions for dealing with polynomials. Notice how each one has the word "polynomial" in the function name like `derivative_of_polynomial`.

Wouldn't it be more elegant if you had a Polynomial class with a `derivative` method? Then you could use your polynomial like this:

```
a = Polynomial([9.0, 0.0, 2.3])
b = Polynomial([-2.0, 4.5, 0.0, 2.1])

print(a, "plus", b , "is", a+b)
print(a, "times", b , "is", a*b)
print(a, "times", 3 , "is", a*3)
print(a, "minus", b , "is", a-b)

c = b.derivative()

print("Derivative of", b ,"is", c)
```

And it would output:

```
2.30x^2 + 9.00 plus 2.10x^3 + 4.50x + -2.00 is 2.10x^3 + 2.30x^2 + 4.50x + 7.00
2.30x^2 + 9.00 times 2.10x^3 + 4.50x + -2.00 is 4.83x^5 + 29.25x^3 + -4.60x^2 +
↪  40.50x + -18.00
2.30x^2 + 9.00 times 3 is 6.90x^2 + 27.00
2.30x^2 + 9.00 minus 2.10x^3 + 4.50x + -2.00 is -2.10x^3 + 2.30x^2 + -4.50x +
↪  11.00
Derivative of 2.10x^3 + 4.50x + -2.00 is 6.30x^2 + 4.50
```

Create a file for your class definition called `Polynomial.py`. Enter the following:

```
class Polynomial:
    def __init__(self, coeffs):
        self.coefficients = coeffs.copy()

    def __repr__(self):
        # Make a list of the monomial strings
        monomial_strings = []

        # For standard form we start at the largest degree
        degree = len(self.coefficients) - 1

        # Go through the list backwards
        while degree >= 0:
            coefficient = self.coefficients[degree]

            if coefficient != 0.0:
                # Describe the monomial
                if degree == 0:
                    monomial_string = "{:.2f}".format(coefficient)
                elif degree == 1:
                    monomial_string = "{:.2f}x".format(coefficient)
```

```
                else:
                    monomial_string = "{:.2f}x^{}".format(coefficient, degree)

                # Add it to the list
                monomial_strings.append(monomial_string)

            # Move to the previous term
            degree = degree - 1

        # Deal with the zero polynomial
        if len(monomial_strings) == 0:
            monomial_strings.append("0.0")

        # Separate the terms with a plus sign
        return " + ".join(monomial_strings)

    def __call__(self, x):
        sum = 0.0
        for degree, coefficient in enumerate(self.coefficients):
            sum = sum + coefficient * x ** degree
        return sum

    def __add__(self, b):
        result_length = max(len(self.coefficients), len(b.coefficients))
        result = []
        for i in range(result_length):
            if i < len(self.coefficients):
                coefficient_a = self.coefficients[i]
            else:
                coefficient_a = 0.0

            if i < len(b.coefficients):
                coefficient_b = b.coefficients[i]
            else:
                coefficient_b = 0.0
            result.append(coefficient_a + coefficient_b)

        return Polynomial(result)

    def __mul__(self, other):

        # Not a polynomial?
        if not isinstance(other, Polynomial):
            # Try to make it a constant polynomial
            other = Polynomial([other])

        # What is the degree of the resulting polynomial?
        result_degree = (len(self.coefficients) - 1) + (len(other.coefficients) -
        ↪  1)

        # Make a list of zeros to hold the coefficients
        result = [0.0] * (result_degree + 1)
```

```
        # Iterate over the indices and values of a
        for a_degree, a_coefficient in enumerate(self.coefficients):

            # Iterate over the indices and values of b
            for b_degree, b_coefficient in enumerate(other.coefficients):

                # Calculate the resulting monomial
                coefficient = a_coefficient * b_coefficient
                degree = a_degree + b_degree

                # Add it to the right bucket
                result[degree] = result[degree] + coefficient

        return Polynomial(result)

    __rmul__ = __mul__

    def __sub__(self, other):
        return self + other * -1.0

    def derivative(self):

        # What is the degree of the resulting polynomial?
        original_degree = len(self.coefficients) - 1
        if original_degree > 0:
            degree_of_derivative = original_degree - 1
        else:
            degree_of_derivative = 0

        # We can ignore the constant term (skip the first coefficient)
        current_degree = 1
        result = []

        # Differentiate each monomial
        while current_degree < len(self.coefficients):
            coefficient = self.coefficients[current_degree]
            result.append(coefficient * current_degree)
            current_degree = current_degree + 1

        # No terms? Make it the zero polynomial
        if len(result) == 0:
            result.append(0.0)

        return Polynomial(result)
```

Create a second file called `test_polynomial.py` to test it:

```
from Polynomial import Polynomial

```

```
a = Polynomial([9.0, 0.0, 2.3])
b = Polynomial([-2.0, 4.5, 0.0, 2.1])

print(a, "plus", b , "is", a+b)
print(a, "times", b , "is", a*b)
print(a, "times", 3 , "is", a*3)
print(a, "minus", b , "is", a-b)

c = b.derivative()

print("Derivative of", b ,"is", c)

slope = c(3)
print("Value of the derivative at 3 is", slope)

```

Run the test code:

```
python3 test_polynomial.py
```

CHAPTER 8

Common Polynomial Products

In math and physics, you will run into certain kinds of polynomials over and over again. In this chapter, we will cover some patterns that you will want to be able to recognize as you move through the sequence!

8.1 Difference of squares

To start off, watch ***Polynomial special products: difference of squares*** from Khan Academy at https://youtu.be/uNweU6I4Icw.

If you are asked what $(3x-7)(3x+7)$ is, you would use the distributive property to expand that to $(3x)(3x)+(3x)(7)+(-7)(3x)+(-7)(7)$. Two of the terms cancel each other, so this is $(3x)^2-(7)^2$. This would simplify to $9x^2-49$

You will see this pattern often. Anytime you see $(a+b)(a-b)$, you should immediately recognize it equals a^2-b^2. (Note that the order doesn't matter: $(a-b)(a+b)$ also equals a^2-b^2.)

Working the other way is important too. Any time you see a^2-b^2, that you should recognize that you can change that into the product $(a+b)(a-b)$. Making something into a product like this is known as *factoring*. You probably have done prime factorization of numbers like $42=2\times3\times7$. In the next couple of chapters, you will learn to factorize polynomials.

Exercise 18 Difference of Squares

Simplify the following products:

1. $(2x-3)(2x+3)$
2. $(7+5x^3)(7-5x^3)$
3. $(x-a)(x+a)$
4. $(3-\pi)(3+\pi)$
5. $(-4x^3+10)(-4x^3-10)$
6. $(x+\sqrt{7})(x-\sqrt{7})$ Factor the following polynomials:
7. x^2-9
8. $49-16x^6$
9. π^2-25x^8
10. x^2-5

Working Space

Answer on Page 61

We are often interested in the roots of a polynomial. That is, we want to know "For what values of x does the polynomial evaluate to zero?" For example, when you deal with falling bodies, the first question you might ask would be "How many seconds before the hammer hits the ground?" Once you have factored a polynomial into binomials, you can easily find the roots.

For example, what are the roots of x^2-5? You just factored it into $(x+\sqrt{5})(x-\sqrt{5})$ This product is zero if and only if one of the factors is zero. The first factor is only zero when x is $-\sqrt{5}$. The second factor is zero only when x is $\sqrt{5}$. Those are the only two roots of this polynomial.

Let's check that result. $\sqrt{5}$ is a little more than 2.2. Using your Python code, you can graph the polynomial:

```
1 import poly
2 import matplotlib.pyplot as plt
3
4 # x**2 - 5
5 pn = [-5.0, 0.0, 1.0]
6
7 # These lists will hold our x and y values
```

```
x_list = []
y_list = []

# Start at x=-3
current_x =-3.0

# End at x=3.0
while current_x < 3.0:
    current_y = poly.evaluate_polynomial(pn, current_x)

    # Add x and y to respective lists
    x_list.append(current_x)
    y_list.append(current_y)

    # Move x forward
    current_x += 0.1

# Plot the curve
plt.plot(x_list, y_list)
plt.grid(True)

#Additional graph customization (make axes visible, axes legend)
plt.axhline(0, color='black', linewidth=1)   # x-axis
plt.axvline(0, color='black', linewidth=1)   # y-axis
plt.title("Graph of y = x² - 5")
plt.xlabel("x")
plt.ylabel("y")

plt.show()
```

You should get a plot like Figure 8.1.

It does, indeed, seem to cross the x-axis near -2.2 and 2.2.

8.2 Powers of binomials

You can raise whole polynomials to exponents. Raising a polynomial to an exponent means taking the entire expression and multiplying it by itself repeatedly according to the value of the exponent. For example,

$$(3x^3+5)^2 = (3x^3+5)(3x^3+5)$$
$$= 9x^6 + 15x^3 + 15x^3 + 25 = 9x^6 + 30x^3 + 25$$

$(3x^3+5)^2$ is a binomial that has been raised to an exponent of 2. A polynomial with two terms is called a *binomial*. $5x^9 - 2x^4$, for example, is a binomial. In this section, we are going to develop some handy techniques for raising a binomial to some power.

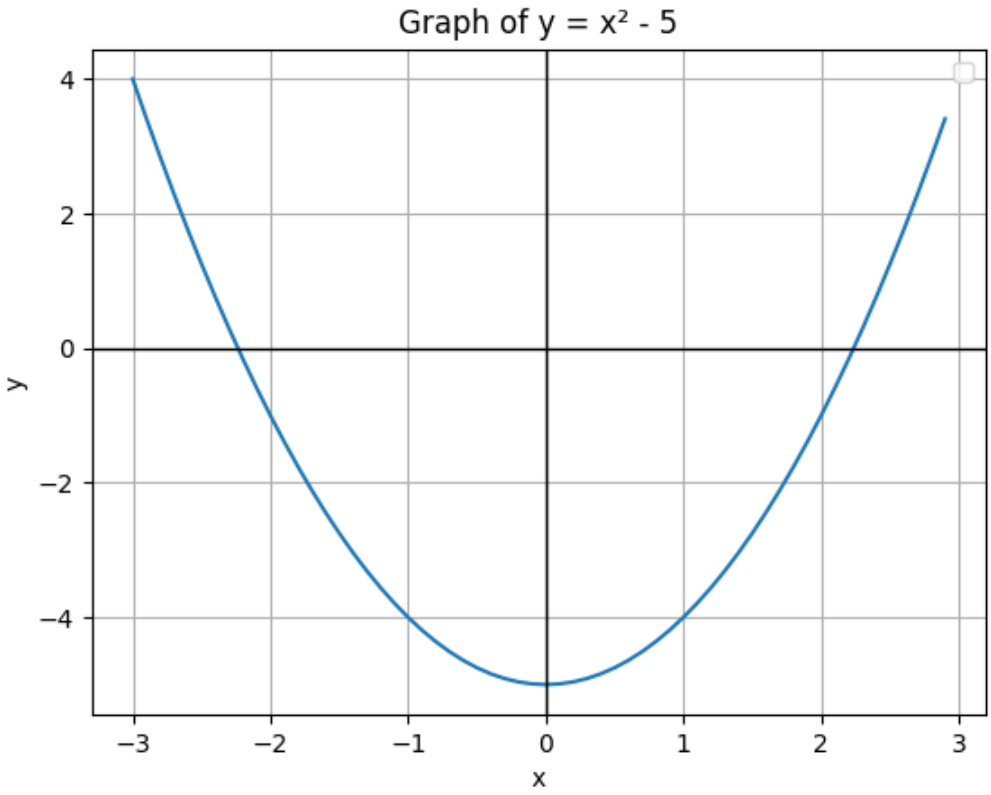


Figure 8.1: A graph of $x^2 - 5$ showing the roots on the x-axis.

Looking at the previous example, you can see that for any monomials a and b, $(a + b)^2 = a^2 + 2ab + b^2$. This is referred to as a perfect square binomial. So, for example, $(7x^3 + \pi)^2 = 49x^6 + 14\pi x^3 + \pi^2$

Exercise 19 Squaring binomials

Simplify the following

1. $(x + 1)^2$
2. $(3x^5 + 5)^2$
3. $(x^3 - 1)^2$
4. $(x - \sqrt{7})^2$

Working Space

Answer on Page 62

What about $(x+2)^3$? You can do it as two separate multiplications:

$$\begin{aligned}(x+2)^3 &= (x+2)(x+2)(x+2)\\ &= (x+2)(x^2+4x+4) = x^3+4x^2+4x+2x^2+8x+8\\ &= x^3+6x^2+12x+8\end{aligned}$$

In general, we can say that for any monomials a and b, $(a+b)^3 = a^3+3a^2b+3ab^2+b^3$.

What about higher powers? $(a+b)^4$, for example? You could use the distributive property four times, but it starts to get pretty tedious.

Here is a trick. This is known as *Pascal's triangle*

$$\begin{array}{ccccccccccc}
 & & & & & 1 & & & & & \\
 & & & & 1 & & 1 & & & & \\
 & & & 1 & & 2 & & 1 & & & \\
 & & 1 & & 3 & & 3 & & 1 & & \\
 & 1 & & 4 & & 6 & & 4 & & 1 & \\
1 & & 5 & & 10 & & 10 & & 5 & & 1 \\
1 & & 6 & & 15 & & 20 & & 15 & & 6 & & 1 \\
1 & & 7 & & 21 & & 35 & & 35 & & 21 & & 7 & & 1 \\
 & & & & & \cdots & & & & &
\end{array}$$

Each entry is the sum of the two above it.

The coefficients of each term are given by the entries in Pascal's triangle:

$$(a+b)^4 = 1a^4+4a^3b+6a^2b^2+4ab^3+1b^4$$

Notice how each term fits the form ax^ky^m. The coefficient a in each term ax^ky^m is known as the binomial coefficient or *binomial theorem*. The binomial coefficient is denoted as $\binom{n}{k}$ where n is the exponent and k is the term number (starting with 0). This is said "n choose k". For example, in $(a+b)^4$, the coefficient of the third term $(6a^2b^2)$ is $\binom{n}{k} = \binom{4}{2} = 6$. We will heavily discuss this in a future chapter on combinatorics.

Exercise 20 Using Pascal's Triangle

Working Space

1. What is $(x+\pi)^5$?

Answer on Page 62

Exercise 21 Coding Pascal's Triangle

Working Space

Write a Python script that finds the row of Pascal's Triangle at a given index n.

Hint: it does not need to be recursive.

Answer on Page 62

Chapter 9

Factoring Polynomials

We factor a polynomial into two or more polynomials of lower degree. For example, let's say that you wanted to factor $5x^3 - 45x$. You would note that you can factor out $5x$ from every term. Thus,

$$5x^3 - 45x = (5x)(x^2 - 9)$$

You might notice that the second factor looks like the difference of squares, so

$$5x^3 - 45x = (5x)(x + 3)(x - 3)$$

That is as far as we can factorize this polynomial.

Why do we care? The factors make it easy to find the roots of the polynomial. This polynomial evaluates to zero if and only if at least one of the factors is zero. Here, we see that

- The factor $(5x)$ is zero when x is zero.
- The factor $(x + 3)$ is zero when x is -3.
- The factor$(x - 3)$ is zero when x is 3.

So, looking at the factorization, you can see that $5x^3 - 45x$ is zero when x is 0, -3, or 3.

This is a graph of that polynomial with its roots circled:

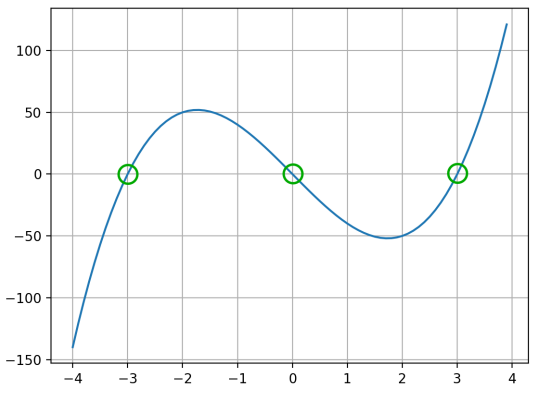


Figure 9.1: Factoring for roots makes it easier to find the roots rather than in expanded form.

9.1 How to factor polynomials

The first step when you are trying to factor a polynomial is to find the greatest common divisor for all the terms, then pull that out. In this case, the greatest common divisor will also be a monomial. Its degree is the least of the degrees of the terms, its coefficient will be the greatest common divisor of the coefficients of the terms.

For example, what can you pull out of this polynomial?

$$12x^{100} + 30x^{31} + 42x^{17}$$

The greatest common divisor of the coefficients (12, 30, and 42) is 6. The least of the degrees of terms (100, 31, and 17) is 17. So, you can pull out $6x^{17}$:

$$12x^{100} + 30x^{31} + 42x^{17} = (6x^{17})(2x^{83} + 5x^{14} + 7)$$

Exercise 22 Factoring out the GCD monomial

FIXME Exercise here

Working Space

Answer on Page 62

So, now you have the product of a monomial and a polynomial. If you are lucky, the polynomial part looks familiar, like the difference of squares or a row from Pascal's triangle.

Often, you are trying factor a quadratic like $x^2 + 5x + 6$ in a pair of binomials. In this case, the result would be $(x + 3)(x + 2)$. Let's check that:

$$(x + 3)(x + 2) = (x)(x) + (3)(x) + (2)(x) + (3)(2) = x^2 + 5x + 6$$

Notice that 3 and 2 multiply to 6 and add to 5. If you were trying to factor $x^2 + 5x + 6$, you would ask yourself"What are two numbers that when multiplied equal 6 and when added equal 5?" And you might guess wrong a couple of times. For example, you might say to yourself, "Well, 6 times 1 is 6. Maybe those work. But 6 and 1 add 7. So those don't work."

Solving these sorts of problems are like solving a Sudoku puzzle. You try things and realize they are wrong, so you backtrack and try something else.

The numbers are sometimes negative. For example, $x^2 + 3x - 10$ factors into $(x+5)(x-2)$.

Exercise 23 Factoring quadratics

Working Space

Answer on Page 62

Chapter 10

Practice with Polynomials

At this point, you know all the pieces necessary to solve problems involving polynomials. In this chapter, you are going to practice using all of these ideas together.

Watch Khan Academy's **Polynomial identities introduction here**: https://youtu.be/EvNKKyhLSpQ Also, watch the follow up here: https://youtu.be/-6qiO49Q180

FIXME: Lots of practice problems here

Appendix A

Answers to Exercises

Answer to Exercise 1 (on page 4)

$\boxed{-2x^3 + \frac{1}{2}x + 3.9}$

$\boxed{(4.5)x^2 + \pi x}$

$\boxed{7}$

$2x^{-10} + 4x - 1$

$x^{\frac{2}{3}}$

$\boxed{3x^{20} + 2x^{19} - 5x^{18}}$

Answer to Exercise 2 (on page 4)

Standard form would be $-x^3 + 21x^2 - 1000x + \pi$. The degree is 3. The leading coefficient is -1

Answer to Exercise 3 (on page 5)

$4^3 - (3)(4^2) + (10)(4) - 12 = 64 - 48 + 40 - 12$. So $y = 44$

Answer to Exercise 4 (on page 8)

```
1 favorites[3] = "Gloves"
```

Answer to Exercise 5 (on page 11)

```
1 pn2 = [2.5, 5.0, -2.0, -7.0, 4.0]
2 y = evaluate_polynomial(pn2, 8.5)
3 print("Polynomia 2: When x is 8.5, y is", y)
4
```

```
pn3 = [-9.0, 0.0, 0.0, 0.0, 0.0, 5.0]
y = evaluate_polynomial(pn3, 2.0)
print("Polynomial 3: When x is 2.0, y is", y)
```

Answer to Exercise 6 (on page 13)

The polynomial crosses the y-axis at 6. When x is zero, all the terms are zero except the last one. Thus, you can easily tell that $x^3 - 7x + 6$ will cross the y-axis at $y = 6$.

Looking at the graph, you tell that the curve crosses the y-axes near -3, 1 and 2. If you plug those numbers into the polynomial, you would find that it evaluates to zero at each one. Thus, $x = -3$, $x = 1$, and $x = 2$ are roots.

Answer to Exercise 7 (on page 16)

$3x^3 - 7x^2 + x - 18$ and $3x^5 - 7x^3 + x^2 - 12$

Answer to Exercise 8 (on page 17)

$x^3 - 3x^2 + 5x$ and $x^5 - 3x^3 + 5x^2 - 2x + 6$

Answer to Exercise 9 (on page 18)

```
def add_polynomials(a, b):
    degree_of_result = max(len(a), len(b))
    result = []
    for i in range(degree_of_result):
        if i < len(a):
            coefficient_a = a[i]
        else:
            coefficient_a = 0.0

        if i < len(b):
            coefficient_b = b[i]
        else:
            coefficient_b = 0.0

        result.append(coefficient_a + coefficient_b)
    return result
```

Answer to Exercise 10 (on page 20)

```
def subtract_polynomial(a, b):
    neg_b = scalar_polynomial_multiply(-1.0, b)
    return add_polynomials(a, neg_b)
```

Answer to Exercise 11 (on page 22)

$(3x^2)(5x^3) = 15x^5$

$(2x)(4x^9) = 8x^{10}$

$(-5.5x^2)(2x^3) = -11x^5$

$(\pi)(-2x^5) = -2\pi x^5$

$(2x)(3x^2)(5x^7) = 30x^{10}$

Answer to Exercise 12 (on page 23)

$(3x^2)(5x^3 - 2x + 3) = 15x^6 - 6x^3 + 6x^2$

$(2x)(4x^9 - 1) = 8x^{10} - 2x$

$(-5.5x^2)(2x^3 + 4x^2 + 6) = 11x^5 - 22x^4 + 33x^2$

$(\pi)(-2x^5 + 3x^4 + x) = -2\pi x^5 + 3\pi x^4 + \pi x$

$(2x)(3x^2)(5x^7 + 2x) = 30x^{10} + 12x^4$

Answer to Exercise 13 (on page 25)

$(2x + 1)(3x - 2) = 6x^2 - x - 2$

$(-3x^2 + 5)(4x - 2) = -12x^3 + 6x^2 + 20x - 10$

$(-2x - 1)(-3x - \pi) = 6x^2 + (4 + 2\pi)x + \pi$

$(-2x^5 + 5x)(3x^5 + 2x) = -6x^{10} + 12x^6 + 10x^2$

Answer to Exercise 14 (on page 25)

The degree of the product is determined by the term that is the product of the highest degree term in p_1 and the highest degree term in p_2. Thus, the product of a degree 23 polynomial and a degree 12 polynomial has degree 35.

Answer to Exercise 15 (on page 32)

1. $f'(t) = 3t^2 - 6t - 4$
2. $g'(t) = (\frac{-3}{4})2t^{-3/4-1} = \frac{-3}{2}t^{-7/4}$
3. $F'(r) = \frac{-15}{r^4}$
4. First, we expand the function by multiplying out the two binomials: $(3u-1)(u+2) = 3u^2 + 6u - u - 2$. Therefore, $H(u) = 3u^2 + 5u - 2$, and we can differentiate using what we have learned about differentiating polynomials. $H'(u) = 6u + 5$. In a later chapter, you will learn the Product rule, which will allow you to differentiate this function without multiplying out the binomials.

Answer to Exercise 16 (on page 33)

```
def derivative_of_polynomial(pn):

    # What is the degree of the resulting polynomial?
    original_degree = len(pn) - 1
    if original_degree > 0:
        degree_of_derivative = original_degree - 1
    else:
        degree_of_derivative = 0

    # We can ignore the constant term (skip the first coefficient)
    current_degree = 1
    result = []

    # Differentiate each monomial
    while current_degree < len(pn):
        coefficient = pn[current_degree]
        result.append(coefficient * current_degree)
        current_degree = current_degree + 1

    # No terms? Make it the zero polynomial
    if len(result) == 0:
        result.append(0.0)
```

```
23
24      return result
```

Answer to Exercise 17 (on page 35)

(a) Velocity is the first derivative of the position function, $s'(t) = 4t^3 - 6t^2 + 2t - 1$. Acceleration is the derivative of the velocity function, $s''(t) = 12t^2 - 12t + 2$.

(b) $s'(1.5) = 4(1.5)^3 - 6(1.5)^2 + 2(1.5) - 1 = 2$ We should note that this is a measurement and needs units to make sense. Since s is in meters and t is in seconds, our velocity should have units of $\frac{m}{s}$, so our final answer is $s'(1.5s) = 2\frac{m}{s}$.

(c) $s''(1.5) = 12(1.5)^2 - 12(1.5) + 2 = 11$. Similarly to part (b), our answer needs units. The units for acceleration are the units for velocity divided by the unit for time (because acceleration is a rate of change of velocity), and our final answer should be $s''(1.5s) = 11\frac{m}{s^2}$.

(d) When velocity and acceleration are occurring in the same direction (i.e. have the same sign), the speed (the absolute value of velocity) is increasing. Since $s'(1.5s)$ and $s''(1.5s)$ are both > 0, the speed of the object is increasing.

Answer to Exercise 18 (on page 46)

$(2x - 3)(2x + 3) = 4x^2 - 9$

$(7 + 5x^3)(7 - 5x^3) = 49 - 25x^6$

$(x - a)(x + a) = x^2 - a^2$

$(3 - \pi)(3 + \pi) = 9 - \pi^2$

$(-4x^3 + 10)(-4x^3 - 10) = 16x^6 - 100$

$(x + \sqrt{7})(x - \sqrt{7}) = x^2 - 7$

$x^2 - 9 = (x + 3)(x - 3)$

$49 - 16x^6 = (7 + 4x^3)(7 + 4^3)$

$\pi^2 - 25x^8 = (\pi + 5x^4)(\pi - 5x^4)$

$x^2 - 5 = (x + \sqrt{5})(x - \sqrt{5})$

Answer to Exercise 19 (on page 48)

$(x+1)^2 = x^2 + 2x + 1$

$(3x^5 + 5)^2 = 9x^10 + 30x^5 + 25$

$(x^3 - 1)^2 = x^6 - 2x^3 + 1$

$(x - \sqrt{7})^2 = x^2 - 2x\sqrt{7} + 7$

Answer to Exercise 20 (on page 49)

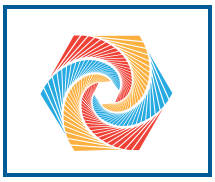
$(x+\pi)^5 = x^5 + 5\pi x^4 + 10\pi^2 x^3 + 10\pi^3 + x^2 + 5\pi^2 x + \pi^5$

Answer to Exercise 21 (on page 50)

```
"""Return the nth row (0-indexed) of Pascal's Triangle."""
n = int(input("Enter row index n: "))
row = [1]
for k in range(1, n + 1):
    row.append(row[-1] * (n - k + 1) // k)
return row
```

Answer to Exercise 22 (on page 52)

Answer to Exercise 23 (on page 53)



Index